

Electric Vehicle Charging Station Problem

Mateusz Adamowicz 284215

27 May 2026

1 Introduction

Electric Vehicle Charging Station Problem (EVCSP) is an optimization problem of choosing locations, number, and types of charging stations for electric cars. In this paper, linear programming and a metaheuristic approach will be evaluated and implemented in order to analyze the problem deeply.

The source code for this project is available at: <https://github.com/Beziki1/Electric-Vehicle-Charging-Station-Problem>.

2 Model

EVCSP can be described by the following mathematical interpretation.

2.0.1 Arguments

Let the grid be defined by dimensions $W \times H$. We define the following sets and parameters:

1. $P_{i,j} \in \{0, 1\}$: POI map, where 1 means a station can be built at (i, j) .
2. $D_{m,n}$: Energy demand at cell (m, n) [kWh].
3. $R_{i,j}$: Land rental cost at cell (i, j) .
4. $d(m, n, i, j)$: Distance between cells (m, n) and (i, j) .
5. $C_{init}^{L2}, C_{init}^{L3}$: Installation costs for Level 2 and Level 3 stations.
6. $C_{maint}^{L2}, C_{maint}^{L3}$: Maintenance costs for Level 2 and Level 3 stations.
7. $P_{power}^{L2}, P_{power}^{L3}$: Power capacity of stations.
8. B : Total available budget.
9. S_{max} : Maximum number of stations per cell.

2.0.2 Decision Variables

1. $x_{i,j} \in \{0, 1\}$: Binary variable, 1 if any station is built at (i, j) .
2. $s_{i,j}^{L2}, s_{i,j}^{L3} \in \mathbb{Z}_{\geq 0}$: Number of L2 and L3 stations at (i, j) .
3. $\alpha_{m,n,i,j} \in [0, 1]$: Fraction of demand from (m, n) assigned to station at (i, j) .

2.0.3 Objective Function

The goal is to minimize the total cost $Z = C_{operator} + C_{users}$:

$$\min Z = \sum_{i,j} (s_{i,j}^{L2} \cdot C_{total}^{L2} + s_{i,j}^{L3} \cdot C_{total}^{L3} + x_{i,j} \cdot R_{i,j}) + \sum_{m,n,i,j} D_{m,n} \cdot \alpha_{m,n,i,j} \cdot d(m, n, i, j) \quad (1)$$

where $C_{total} = C_{init} + C_{maint}$.

2.0.4 Constraints

1. **Demand Satisfaction:** All demands must be fully met.

$$\sum_{i,j} \alpha_{m,n,i,j} = 1, \quad \forall(m, n) \quad (2)$$

2. **Capacity:** Total demand allocated to a station cannot exceed its capacity.

$$\sum_{m,n} D_{m,n} \cdot \alpha_{m,n,i,j} \leq (s_{i,j}^{L2} \cdot P_{power}^{L2} + s_{i,j}^{L3} \cdot P_{power}^{L3}) \cdot 24, \quad \forall(i, j) \quad (3)$$

3. **POI and Construction:** Stations can be built only at POIs and within limits.

$$s_{i,j}^{L2} + s_{i,j}^{L3} \leq S_{max} \cdot x_{i,j}, \quad \forall(i, j) \quad (4)$$

$$x_{i,j} \leq P_{i,j}, \quad \forall(i, j) \quad (5)$$

4. **Budget:** Total investment cannot exceed B .

$$\sum_{i,j} (s_{i,j}^{L2} \cdot C_{init}^{L2} + s_{i,j}^{L3} \cdot C_{init}^{L3} + x_{i,j} \cdot R_{i,j}) \leq B \quad (6)$$

2.0.5 Implementation

The mathematical model described above was implemented as a Mixed-Integer Linear Programming (MILP) problem. The solution was obtained using the Gurobi Optimizer [2], leveraging its high-performance branch-and-cut algorithms to ensure global optimality within the defined constraints.

3 Metaheuristic Model

To find solutions for large-scale problem instances where classical Mixed-Integer Linear Programming (MILP) becomes computationally inefficient, a metaheuristic algorithm based on *Bacterial Foraging Optimization* (BFO) was implemented. This algorithm, developed based on the theoretical foundations and applications described by Das et al. [1], is inspired by the foraging behavior of *E. coli* bacteria, utilizing mechanisms of chemotaxis, swarming, reproduction, and elimination-dispersal.

3.0.1 Representation and Population

In the model, each bacterium represents a single potential solution to the EVCSP. The state of a bacterium is defined by the matrices of station locations $s_{i,j}^{L2}$ and $s_{i,j}^{L3}$. The fitness function corresponds to the objective function value Z , augmented by penalty terms for violating budget constraints or failing to satisfy total energy demand.

3.0.2 Algorithmic Steps

The optimization process consists of four primary nested loops:

1. **Chemotaxis:** This phase consists of a *tumble* (changing direction through random modification of station counts in POI cells) and a *swim* (continuing movement in a direction that yields a lower cost).
2. **Swarming:** A communication mechanism between bacteria. It uses attraction and repulsion functions to steer the population toward promising areas while preventing premature convergence to local minima:

$$J_{sw} = \sum_{i=1}^N [-d_{attr} \exp(-w_{attr} \cdot dist^2) + h_{rep} \exp(-w_{rep} \cdot dist^2)] \quad (7)$$

3. **Reproduction:** After the chemotactic stages, the population is sorted based on cumulative "health" (fitness over its lifetime). The least fit half of the population dies, while the healthier half undergoes cell division (cloning) to maintain a constant population size.
4. **Elimination and Dispersal:** Individuals are removed and replaced by entirely new random solutions with a probability P_{ed} . This ensures global exploration and prevents the algorithm from getting trapped in stagnant regions of the search space.

3.0.3 BFO Algorithm Pseudocode

The following pseudocode outlines the structure of the implemented `MetaheuristicModel`.

Algorithm 1: Bacterial Foraging Optimization for EVCSP

Result: Best solution (Bacterium with minimal cost)
Initialize population of N bacteria randomly within valid POIs;
for $l = 1$ **to** N_{ed} (*Elimination-Dispersal Loop*) **do**
 for $k = 1$ **to** N_{re} (*Reproduction Loop*) **do**
 for $j = 1$ **to** N_{ch} (*Chemotaxis Loop*) **do**
 foreach Bacterium $i \in \{1, \dots, N\}$ **do**
 Compute Swarming Effect $J_{sw}(i)$;
 $J(i, j, k, l) = \text{Evaluate}(i) + J_{sw}(i)$;
 Tumble: Randomly modify $s_{i,j}$ and validate constraints;
 Swim: **while** *swimming steps* $< N_s$ **and** *fitness improves* **do**
 Move in the same direction;
 Update health and cost values;
 end
 end
 end
 Reproduction: Sort by health, clone the best half, remove the rest;
 end
 Elimination-Dispersal: Replace random individuals with probability P_{ed} ;
end

3.0.4 Modifications

In order to properly train this BFO model following modifications were made:

1. **Change of decision variables:** Station locations, L2 station locations, L3 station locations and demand allocations map were all decision variables in MILP model. This approach in the metaheuristic one were proven to be inefficient. Instead BFO [1] learns only L2 and L3 station maps. Demand allocation map is calculated dynamically using greedy approach to find closest station to the given demand point. Station locations result from L2 and L3 maps.
2. **Repair System:** Model needs to repair itself after each change of Bacterium position. This operation is necessary to stick to the constraints of the problem.

4 Comparison

In this section, the performance of the Mixed-Integer Linear Programming (MILP) approach using the Gurobi Optimizer is compared against the Bacterial Foraging Optimization (BFO) metaheuristic. The comparison focuses

on the solution quality (objective function value) and computational efficiency across varying grid sizes.

4.1 Experimental Results

The models were tested on synthetic datasets representing different urban scales. The results are summarized in Table 1.

Table 1: Comparison of MILP (Gurobi) and Metaheuristic (BFO) results.

ID	Grid (W×H)	BFO Cost	MILP Cost	Gap (%)	BFO Time	MILP Time
1	10×10	316808.00	313432.00	1.07%	0.266 s	0.092 s
2	20×20	675200.00	672000.00	0.47%	1.483 s	7.053 s
3	30×30	4010700.00	3774600.00	5.89%	76.265 s	Timeout

Timeout - model cant find solution in reasonable time, so data are taken from up to 5000 s.

4.2 Conclusion

The analysis reveals a significant trade-off between optimality and scalability. The MILP approach, while guaranteeing the global optimum, exhibits exponential growth in computational complexity as the grid size increases. For large-scale urban planning (100×100 and above), MILP becomes practically infeasible due to memory and time constraints.

Conversely, the Bacterial Foraging Optimization (BFO) metaheuristic provides a robust alternative. Although it does not guarantee global optimality, the results show that it consistently achieves solutions with an optimality gap below 6% compared to MILP. The primary advantage of BFO is its computational efficiency; it delivers near-optimal solutions significantly faster than MILP for large datasets, making it the preferred choice for real-time or large-scale EV infrastructure planning.

5 Bibliography

- [1] Swagatam Das, Arijit Biswas, Sambarta Dasgupta, and Ajith Abraham. Bacterial foraging optimization algorithm: Theoretical foundations, analysis, and applications. In *Foundations of Computational Intelligence Volume 3: Theoretical Foundations and Applications*, volume 203 of *Studies in Computational Intelligence*, pages 23–55. Springer-Verlag, Berlin, Heidelberg, 2009.
- [2] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*, 2026. Version 11.0.